A Place to Start with Objects

I guess this all began with a pile of tiny Java files and a simple question: *what is OOP, really?* Not the textbook checklist. Not the four fancy words people wave around in interviews. Just… how do you look at a program and see *things* that know how to do stuff?

That's what this little guide is for.

You've already got working code in this project. We're going to walk through it the way a beginner should meet it: one idea at a time, with your actual files as the map. If something feels weird at first, that's normal. Learning to program is hard enough as it is — we don't need to make it harder by dumping every concept on you at once.

By the end, you'll know what a **class** is, what an **object** is, why we hide some details, how one kind of thing can be a more specific kind of another thing, how a parent can stay deliberately unfinished (**abstract**), and how Java can treat different payment methods as "the same kind of thing" when that helps. Then there's a small exercise waiting for you.

Ready? Let's start somewhere friendly.

Thoughts Before We Dive In

A few guiding ideas for this guide (and for you, if you ever explain this to someone else):

1. **One concept at a time.** We'll meet cars before we meet inheritance. We'll meet a username before we meet PayPal.
2. **One clear way to do something.** There are other ways. We won't chase them yet. Creativity grows when you have fewer tools and a real problem.
3. **Intuition first, jargon second.** I'll talk about cars and dogs and payments — the things in these programs — and only then give the official names, so you learn the *right* word once.
4. **Your code is the textbook.** The samples below match what's in `src/main/java/com/mycompany/java_oop/`. Open those files while you read. The comments in the code and this guide should feel like they're talking to each other.

Alright. First stop: a car.

---

1. Classes and Objects — Your First Car

Open `Car.java`.

A **class** is a blueprint. A **object** is one real thing built from that blueprint.

Think of it like a cookie cutter and a cookie. The cutter says "this is the *shape* of a cookie." The cookie is the thing you can actually eat. In our program, `Car` is the cutter. When we write `new Car()`, we get one cookie — one car sitting in memory, ready to drive.

Here's the idea, stripped down:

```
public class Car {

    int speed;

    String model;



    void drive() {

        System.out.println("Driving ...");

    }

}
```

Those `speed` and `model` bits are the car's **fields** — information the car can hold. The `drive()` bit is a **method** — something the car can *do*.

In `Main.java` we actually build one:

```
Car car1 = new Car();

car1.drive();
```

Read that out loud: "Make a new Car, call it `car1`, and ask it to drive."

That's OOP in one breath. You create a thing. You ask the thing to do something.

Try this: Make a second car — `Car car2 = new Car();` — and call `drive()` on both. Same blueprint, two separate cars.

---

2. Constructors and Encapsulation — Saying Hello to a User

Open `User.java`.

A car with no model is a bit vague. A user with no name is worse. So when we create a `User`, we hand in a name right away:

```
public User(String username) {

    this.username = username;

}
```

That special method with the same name as the class is a **constructor**. It runs when you say `new User("Harsha")`. It's the moment the object is born and given its starting details.

Notice something else:

```
private String username;
```

`private` means: *hey outside world, you don't get to poke this field directly.* If you want the username, you go through the methods the class gives you — like `getUser()`.

That habit — keeping the insides tucked away and only showing a careful doorway — is called **encapsulation**. Fancy word, simple idea: the object looks after its own data.

In `Main.java`:

```
var user = new User("Harsha");

user.getUser();
```

`var` just means "Java, you figure out the type — it's obviously a User." Same as writing `User user = new User("Harsha");`.

Why bother with private? Imagine later you want usernames to always be lowercase, or never empty. If everyone can reach into `username` like a public drawer, you can't enforce that. If they must go through your methods, you *can*.

---

3. Inheritance — Animals, and Then Dogs

Open `Animal.java` and `Dog.java`.

Some animals eat. Dogs eat *and* make a sound. It would be silly to copy the eating code into every animal type. Instead, we say: a Dog **is an** Animal — with extras.

```
public abstract class Animal {

    protected String breed;


    public Animal(String breed) {

        this.breed = breed;

    }


    void eat() {

        System.out.println(this.breed + " Eating ...");

    }


    abstract void makeSound();

}
```

(Don't panic about `abstract` yet — that's the next section. For now, notice the family relationship.)

`protected` is a middle ground between `private` and "everyone can see it." Subclasses (like `Dog`) can use `breed`. Random unrelated classes should not.

Now the dog:

```
public class Dog extends Animal {

    public Dog(String breed) {
```

```
        super(breed);   // hand the breed up to Animal's constructor
    }

    @Override
    void makeSound() {
        System.out.println(this.breed + " Barking ...");
    }
}
```

`extends` means: start with everything Animal has, then add what's special about dogs.

`super(breed)` means: "Parent constructor, please set this up the way you know how." Dogs don't reinvent birth certificates; they reuse Animal's.

In `Main.java`:

```
var dog = new Dog("German Shepard");
dog.eat();         // came from Animal
dog.makeSound();  // filled in by Dog
```

One object. Two layers of behavior. That's **inheritance**.

Picture it:
Animal
└── Dog

Dogs get eating for free. The sound is their own thing.

---

4. Abstract Classes — An Unfinished Blueprint on Purpose

Still in `Animal.java` — look at that word again: `abstract`.

Sometimes a parent class is useful as a *shared starting point*, but it would be weird to create one of those parents by itself. What does a plain "Animal" sound like? You don't know. A dog barks. A cat meows. "Animal" is the idea; dogs and cats are the real things.

So we mark the class `abstract`:

```
public abstract class Animal {

    // ...

}
```

That means: **you cannot do** `new Animal("Something")`. Java will refuse. You can only `new` a concrete child, like `new Dog("German Shepard")`.

And we can leave a method unfinished on purpose:

```
abstract void makeSound();
```

No curly braces. No body. It's a homework assignment for every subclass: "You must write `makeSound()` before you're a complete animal."

`Dog` does the homework with `@Override`. If you later add a `Cat`, it must write its own `makeSound()` too — or Java won't let that class compile. That's the point. The abstract parent *forces* the important parts to be filled in, while still sharing the parts that are the same (`eat()`, the `breed` field, the constructor).

Abstract class vs interface (quick gut check):

- **Abstract class** — "You *are* a kind of this, and here's some ready-made code you inherit." (`Dog extends Animal`)
- **Interface** — "You *can do* this job; I won't give you any shared fields/constructor here." (`PayPal implements Payment`)

---

5. Interfaces and Polymorphism — Paying in More Than One Way

Open `Payment.java`, `PayPal.java`, and `Stripe.java`.

Sometimes you don't care *how* something works — only that it can do a job. Paying is a job. PayPal and Stripe are two different ways to do that job.

An **interface** is a promise. It says: "If you claim to be a Payment, you must have a `pay()` method." It does *not* say how you pay.

```
public interface Payment {

    void pay();

}
```

PayPal keeps the promise one way:

```
public class PayPal implements Payment {

    @Override

    public void pay() {

        System.out.println("Paid through PayPal");

    }

}
```

Stripe keeps it another way:

```
public class Stripe implements Payment {

    @Override

    public void pay() {

        System.out.println("Paid through Stripe");

    }

}
```

`implements` = "I promise to fill in the methods from this interface."
`@Override` = "Yes, this is me fulfilling that promise (or replacing a parent method)."

Now the fun part — in `Main.java`:

```
Payment payment1 = new PayPal();

Payment payment2 = new Stripe();
```

```
payment1.pay();

payment2.pay();
```

Look carefully. The *variable type* is `Payment`. The *actual objects* are PayPal and Stripe. You call `pay()` on both, and each one does its own thing.

That's **polymorphism**: same message (`pay()`), different behavior depending on what you actually built. You can write code that talks to "a Payment" without caring which company is behind it. Later, if you add Apple Pay, the calling code barely has to change.

One way to remember it: Inheritance is "is a more specific kind of." Interfaces are "can do this job." Dogs *are* animals. PayPal *can* pay.

---

6. Putting It Together — `Main.java`

`Main.java` is the stage. Everything we've met walks on for a moment:

1. Build a car, ask it to drive.
2. Build a user named Harsha, print the name.
3. Build a German Shepherd from the abstract Animal family; let it eat and make a sound.
4. Build two payments (PayPal and Stripe), call `pay()` through the `Payment` type.

Run the project and you should see something like:

```
Driving ...

User Name = Harsha

German Shepard Eating ...

German Shepard Barking ...

Paid through PayPal

Paid through Stripe
```

If that prints, your mental model is working: blueprints, newborns with constructors, unfinished parents (abstract classes), shared behavior through inheritance, and shared *jobs* through interfaces.

---

7. Your Turn — The Browser Exercise

Open `BrowserExercise.java`. There's a challenge waiting there (and in the comment block at the top of the file).

Don't just define the OOP words. *Use* them.

Build a tiny browser system:

```
Browser
  ├── Chrome
  └── Firefox
```

- `Browser` should be an interface (or abstract idea) with `start()`.
- Chrome prints: `Chrome started`
- Firefox prints: `Firefox started`
- In `main()`, demonstrate polymorphism:

```
Browser browser = new Chrome();

browser.start();



browser = new Firefox();

browser.start();
```

Write the whole thing yourself. This one exercise hits interfaces, implementation, objects, methods, and polymorphism in one shot — which is honestly a nicer way to learn than memorizing a list.

When you're stuck, re-read section 5. Payments and browsers are the same shape with different names. Feeling brave? Try `Browser` as an *abstract class* with a shared method plus an abstract `start()` — same idea as Animal.

Did We Cover "All" of OOP?

For a beginner Java tour: **yes, the main toolkit is here.**

Idea	Where you met it
Classes & objects	Car
Encapsulation	User + private
Inheritance	Dog extends Animal
Abstract classes	abstract class Animal + makeSound()
Interfaces	Payment
Polymorphism	Payment p = new PayPal() / Stripe

Those map to the classic four big ideas people name (**encapsulation, inheritance, polymorphism, abstraction**) — abstraction shows up in both abstract classes and interfaces: "hide the messy how, show a clear what."

There's always more later (composition, overriding vs overloading, access tricks, design patterns…). You don't need them to say you've stood on solid OOP ground. Finish the browser exercise, then go explore when you're curious.

A Tiny Glossary (Only When You Need It)

Word	Plain meaning in this project
Class	Blueprint (Car, User, Dog…)
Object	One living thing made with new
Field	Data the object holds (username, breed)
Method	Something the object can do (drive(), pay())
Constructor	Setup code that runs on new
Encapsulation	Keep data private; offer careful methods
Inheritance	Dog extends Animal — reuse + specialize

Abstract class	Unfinished parent you can't new (Animal)
Interface	A job description (Payment has pay())
Polymorphism	Same call, different real behavior

You don't need to recite these. You need to *recognize* them when you see them in your files.

Where to Go Next

If this clicked, try stretching the same muscles:

- Add a `Cat` that extends `Animal` and implements `makeSound()` with a meow.
- Add a third payment class (your own made-up company is fine).
- Finish the browser exercise, then try `Safari` too.
- Give `Car` a constructor that sets `model` and `speed`, then print them from `drive()`.

Boring exercises kill the desire to program. Good ones create an itch you can't help scratching. So pick the itch that sounds fun — and scratch it in code.

If something in this guide doesn't match what you see when you run the program, trust the program, then come fix the words. The code is the truth. This file is just a friendly tour guide walking beside it. :-)